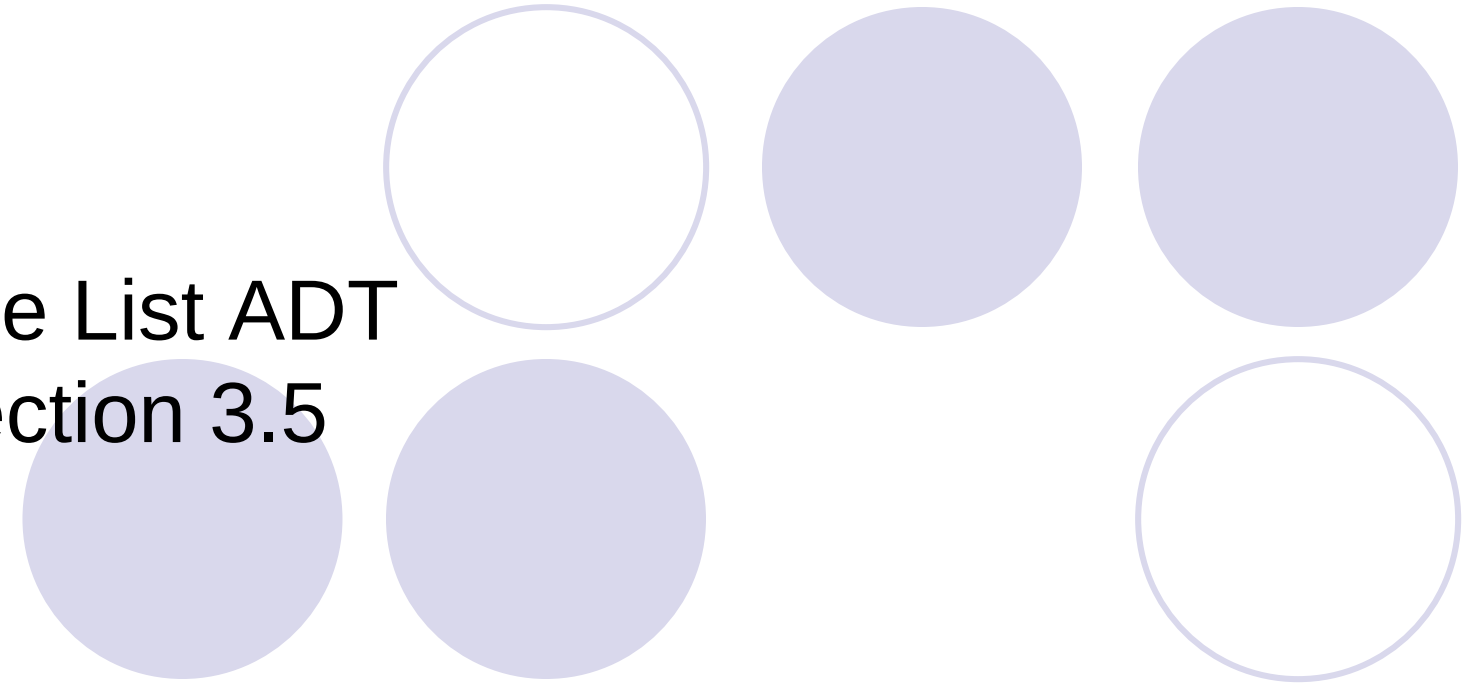
The List ADT
Section 3.5



List ADT

- Definition:
  - $A_0, A_1, A_2, \ldots, A_{N-1}$
  - size of this list is N.
  - list of size 0 is an **empty list**
  - The position of element $\mathbf{A}_i$ in a list is ***i***.
- Operations:
  - **printList, makeEmpty, find, insert, next, previous, …etc**

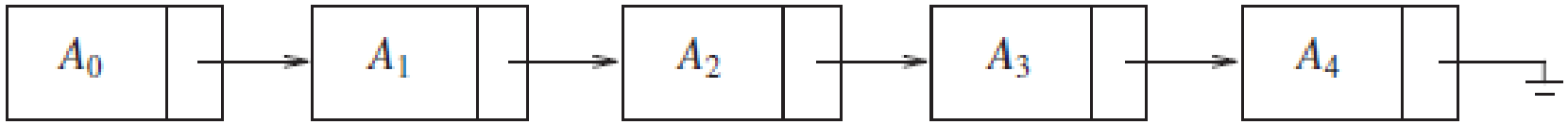
List Implementation

- Simple Array

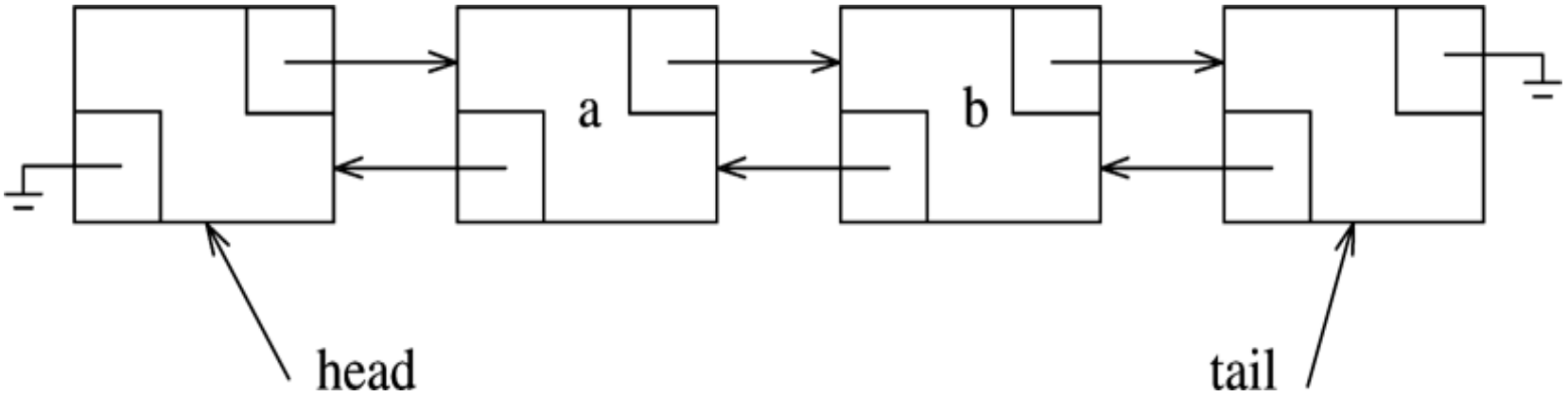
  $O(1)$: find(x), insert and delete at the end

  $O(N)$: insert and delete

- Simple Linked Lists

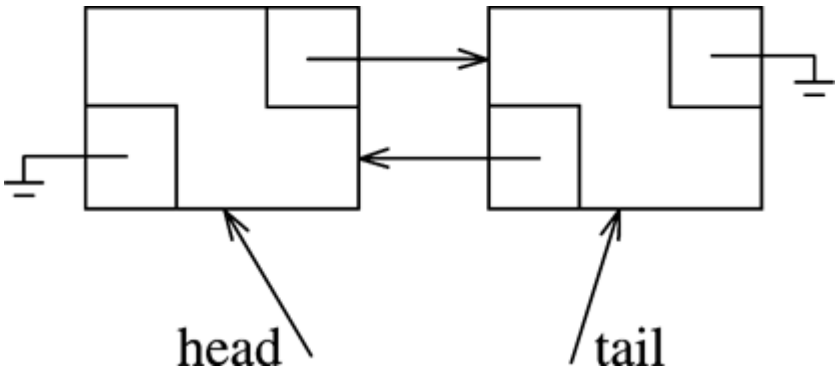


List Implementation



A Doubly Linked List With Header and Tail Nodes as Markers (sentinels)

An Empty List



List Complexity Requirements

- *O(1)* : Constant runtime complexity
  - Default Constructor
  - **`push_front(t), push_back(t), insert(I, t)`**
  - **`pop_front(), pop_back(), erase(I)`**
  - **`begin(), end();`**
  - **`front(), back();`**
  - **`empty();`**

List Complexity Requirements (2)

- *O(N)* : Linear runtime complexity
  - Copy Constructor
  - Destructor
  - **`clear()`**
  - **`erase(SI,EI)`**

Using List

```
List<String> DSARoster;

DSARoster.push_front("Ahmad");
```

- "Ahmad"

```
DSARoster.push_back("Mohammad");
```

- "Ahmad", "Mohammad"

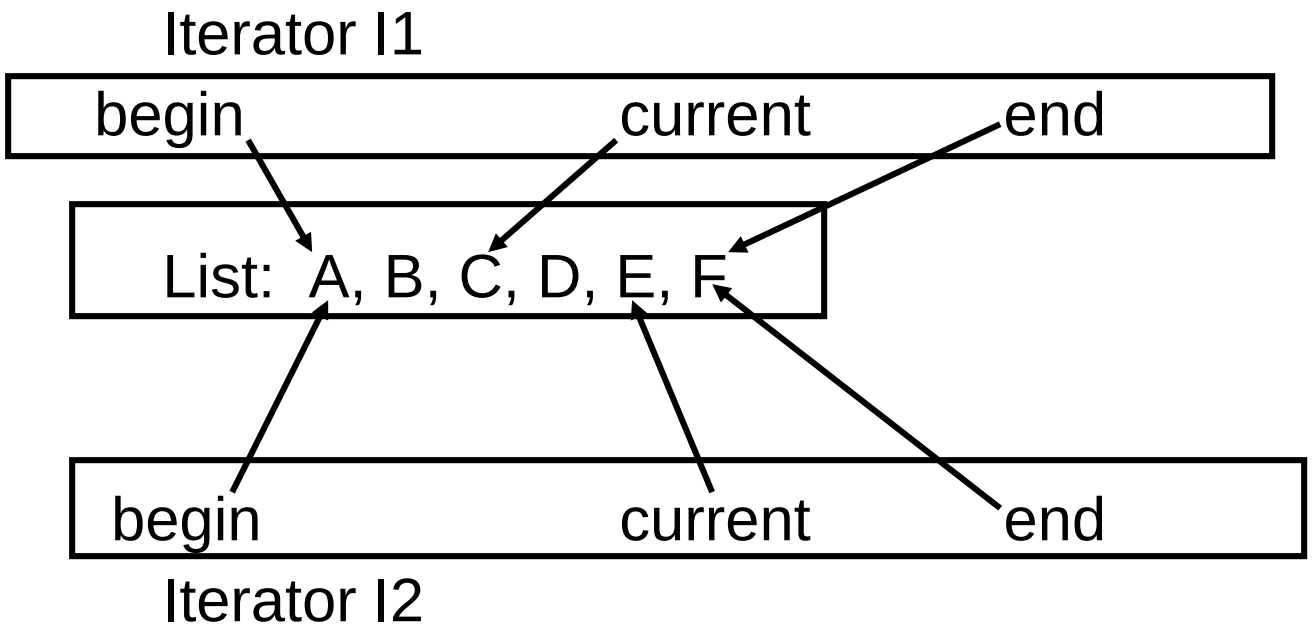
```
List<String>::iterator I = DSARoster.begin();
DSARoster.insert(I,"Abdullah");
```

- "Abdullah", "Ahmad", "Mohammad"

```
for (I = DSARoster.begin(); I != DSARoster.end(); I++) {
   // print list with <<
}
```

List and List Iterator

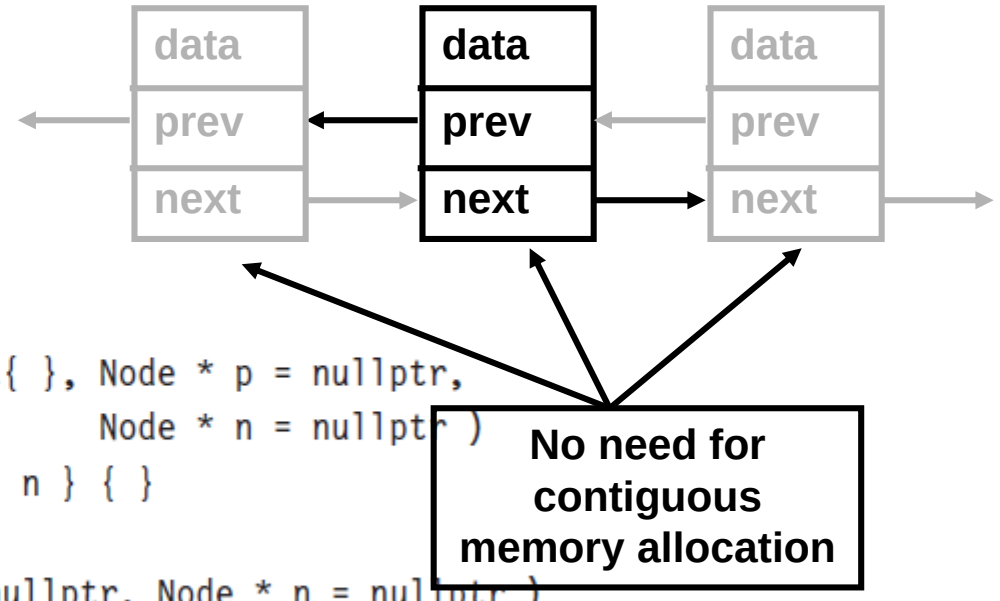
- Conceptual relationship



Nodes in a list

- Node
  - Pointers to the previous and next element
  - Data Value
- Defined within the List class, with limited scope

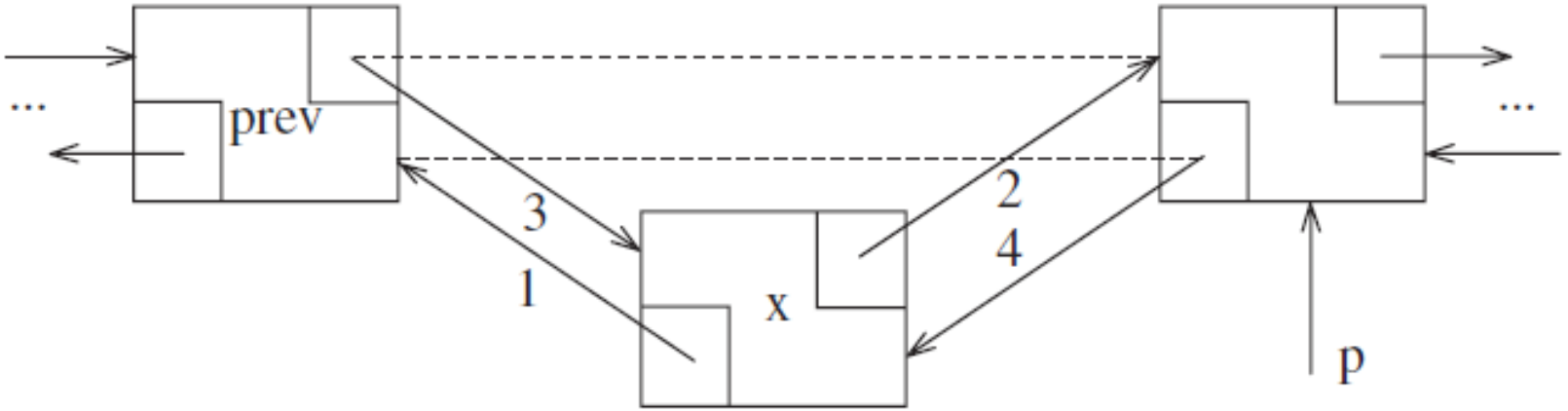
```
1     struct Node
2     {
3         Object  data;
4         Node   *prev;
5         Node   *next;
6
7         Node( const Object & d = Object{ }, Node * p = nullptr,
8                                             Node * n = nullptr )
9           : data{ d }, prev{ p }, next{ n } { }
10
11        Node( Object && d, Node * p = nullptr, Node * n = nullptr )
12          : data{ std::move( d ) }, prev{ p }, next{ n } { }
13    };
```



Implementation

- List class outline: Figures 3.11 and 3.12
- Node class: Figure 3.13
- const_iterator class: Figure 3.14
- iterator class: Figure 3.15
- List constructor, Big-five, init: Figure 3.16

insert




```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev->next = newNode;                    // Step 3
p->prev = newNode;                          // Step 4
```

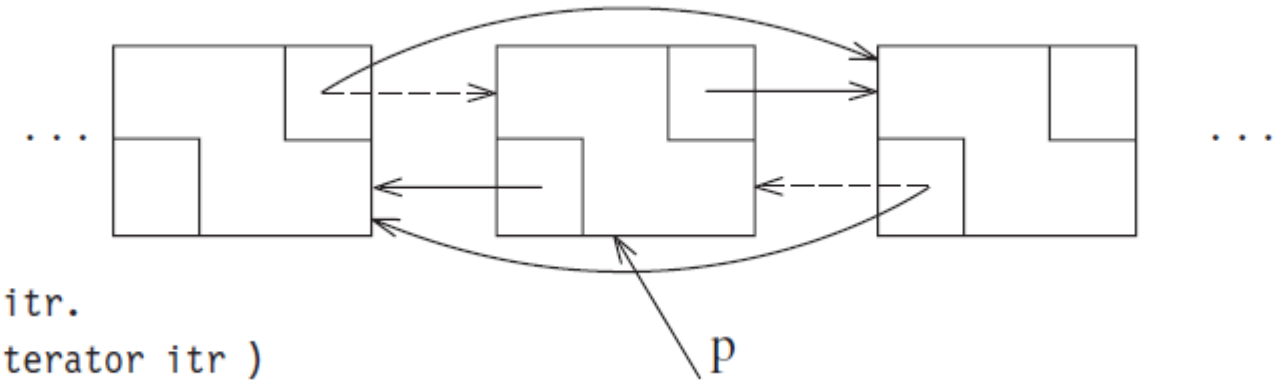
```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev = p->prev->next = newNode;          // Steps 3 and 4
```

```
p->prev = p->prev->next = new Node{ x, p->prev, p };
```

insert

```
1       // Insert x before itr.
2       iterator insert( iterator itr, const Object & x )
3       {
4           Node *p = itr.current;
5           theSize++;
6           return { p->prev = p->prev->next = new Node{ x, p->prev, p } };
7       }
8
9       // Insert x before itr.
10      iterator insert( iterator itr, Object && x )
11      {
12          Node *p = itr.current;
13          theSize++;
14          return { p->prev = p->prev->next
15                                  = new Node{ std::move( x ), p->prev, p } };
16      }
```

erase




```
1       // Erase item at itr.
2       iterator erase( iterator itr )
3       {
4           Node *p = itr.current;
5           iterator retVal{ p->next };
6           p->prev->next = p->next;
7           p->next->prev = p->prev;
8           delete p;
9           theSize--;
10
11          return retVal;
12      }
13
14      iterator erase( iterator from, iterator to )
15      {
16          for( iterator itr = from; itr != to; )
17              itr = erase( itr );
18
19          return to;
```

```
p->prev->next = p->next;
p->next->prev = p->prev;
delete p;
```